

A DAVE-ML Flight-Model Tabulation for the Aetherion Ducted-Fan Tail-Sitter

Structure, Conventions, Provenance, and Declared Limits

Onur Tuncer*

Çağlar Üçler†

Ahmet Güneş‡

15 August 2026

Aeolion Technical Report ATR-2026-01

Abstract

This report specifies a single-file DAVE-ML flight model—the encoding of the American National Standard ANSI/AIAA S-119-2011—for the Aetherion ducted-fan tail-sitter, assembled from the Aeolion aerodynamic toolkit. It states the model’s structure, its reference-frame and sign conventions, the nondimensionalisation and buildup of every tabulated quantity, the provenance of each of its 36 tables, and—given equal weight—the boundaries of what the underlying methods can support. Three structural decisions carry the design. First, the model is one file rather than the conventional aerodynamics/propulsion pair, because the power-on interaction increments depend on breakpoints owned by both halves and DAVE-ML provides no cross-file reference mechanism. Second, forward speed and rotor speed are collapsed into the advance ratio J , which is exact for the inviscid rotating-frame solve that generates the propulsor tables. Third, the model declares *no* Mach and *no* Reynolds axis: the toolkit is incompressible throughout and every source sweep was run at a single flight condition, so such axes would encode a fidelity that does not exist. The tabulation is verified not by inspection but by generated `checkData` static shots evaluated through an in-repo interpreter, including a moment-transfer shot that would detect the specific frame-conversion error that has occurred once in this codebase’s history. Two blocks of the model are declared incomplete and are identified as such in the file itself: the fan-on-airframe interaction increments await an upstream-induction model, and the propulsor tables are axial-inflow only.

Contents

Nomenclature	3
1 Purpose and scope	4
1.1 Standards conformance	4
1.2 What is deliberately absent	5
2 Model architecture	6
2.1 One file, not two	6
2.2 Structure at a glance	6

*Professor, Department of Aeronautical Engineering, Istanbul Technical University; onur.tuncer@itu.edu.tr

†Associate Professor, Özyeğin University; caglar.ucler@ozyegin.edu.tr

‡Associate Professor, Istanbul Technical University

3	Reference frames, signs, and the moment reference	6
3.1	The two frames	6
3.2	A recorded failure, and the check that now catches it	7
3.3	Moment reference point	7
3.4	Deflection sign convention	7
4	Nondimensionalisation	8
4.1	Drag accounting	8
5	Variables	10
5.1	Inputs	10
5.2	Constants	10
5.3	Derived variables	10
6	The vane command space	11
6.1	Why not four independent angles	11
6.2	Mode shapes from the geometry	11
6.3	Three properties the tables must respect	12
6.4	Travel limits	12
7	Breakpoint sets	12
8	Table inventory and provenance	13
8.1	aero*: airframe, power-off	13
8.2	prop*: rotor, duct, and vanes	15
8.3	Axial inflow only: a declared structural limit	15
8.4	coupling*: fan-on-airframe increments	15
9	The buildup	16
10	First generated results	17
10.1	Parasite drag, and the case against a constant	18
10.2	Vane-mode superposition: measured, and partly false	18
10.3	Three conditions that do not converge, and what was measured about them . . .	21
11	Verification	22
11.1	Static shots	22
11.2	The evaluator	23
12	Generation pipeline	23
13	Declared limits	23
14	Open items	24

Nomenclature

AR	= wing aspect ratio, b^2/S
b	= wing reference span, m
$\bar{c}$	= wing reference chord, m
C_D	= total drag coefficient
C_{D_0}	= body and duct parasite drag coefficient
C_{D_i}	= induced drag coefficient
c_d	= section profile drag coefficient
C_L	= lift coefficient
$C_{L\max}$	= maximum lift coefficient
C_l, C_m, C_n	= rolling, pitching, yawing moment coefficients, body axes
C_{l_p}, C_{l_r}	= roll damping and roll-due-to-yaw-rate derivatives, per $\hat{p}, \hat{r}$
C_{m_q}	= pitch damping derivative, per $\hat{q}$
C_{n_p}, C_{n_r}	= yaw-due-to-roll-rate and yaw damping derivatives
C_N	= normal-force coefficient, chord-plane normal
C_Q	= rotor torque coefficient, $Q/\rho n^2 D^5$
C_T	= rotor thrust coefficient, $T/\rho n^2 D^4$
C_X, C_Y, C_Z	= body-axis force coefficients
$C_{d\max}$	= deep-stall section drag maximum (Viterna anchor)
D	= rotor diameter, m
D_i	= induced drag, N
f	= Kirchhoff attached-flow fraction, chordwise separation point
F_x, F_y, F_z	= total body-axis forces, N
J	= advance ratio, V_∞/nD
M	= Mach number
M_x, M_y, M_z	= total body-axis moments about the reference point, N m
n	= rotor rotational speed, rev/s
p, q, r	= body-axis roll, pitch, yaw rates, rad/s
$\hat{p}, \hat{q}, \hat{r}$	= reduced (nondimensional) body rates
$\bar{q}$	= free-stream dynamic pressure, $\frac{1}{2}\rho V_\infty^2$, Pa
Q	= rotor shaft torque, N m
Re_n	= leading-edge-normal Reynolds number
r_{LE}	= section leading-edge radius, m
S	= wing gross planform reference area, m ²
s	= swirl factor of the rotor-vane momentum budget
T	= rotor thrust, N
T_c	= airframe-normalised thrust coefficient, $T/\bar{q}S$
$\hat{u}$	= free-stream unit vector, body axes
V_∞	= true airspeed, m/s
v_i	= axial induced velocity, m/s
x, y, z	= body-axis coordinates, m
x_{cp}	= centre of pressure, fraction of chord
α	= angle of attack, deg
α_{disk}	= angle between the free stream and the rotor axis, deg
β	= sideslip angle, deg
δ_a	= aileron deflection (right surface), deg
$\delta_P, \delta_Y, \delta_R$	= vane pitch, yaw, roll mode commands, deg
$\delta_b, \delta_l,$ δ_t, δ_r	= individual vane deflections (bottom, left, top, right), deg
η	= span fraction (wing) or exit-radius fraction (vanes)

ρ	= air density, kg/m ³
σ	= total inclination of the free stream to the chord plane, $\sin \sigma = \sin \alpha \cos \beta$, deg
ϕ_w	= crossflow azimuth about the rotor axis, deg
Ω	= rotor angular speed, rad/s

Subscripts

<i>aero</i>	airframe, power-off contribution
<i>cpl</i>	fan-on-airframe interaction contribution
<i>disk</i>	referred to the rotor disk
<i>frd</i>	contract body frame: x forward, y right, z down
<i>mrp</i>	moment reference point
<i>prop</i>	propulsor contribution
<i>vlm</i>	solver frame: x aft, y right, z up
∞	free-stream condition

Abbreviations

<i>BP</i>	breakpoint set
<i>CST</i>	class/shape transformation airfoil parameterisation
<i>DAVE-ML</i>	Dynamic Aerospace Vehicle Exchange Markup Language (AIAA S-119)
<i>FRD</i>	forward–right–down body axes
<i>MRP</i>	moment reference point
<i>RMS</i>	root mean square
<i>VLM</i>	vortex-lattice method

1 Purpose and scope

The Aeolion toolkit computes the aerodynamics of a ducted-fan tail-sitter by a family of related potential-flow methods: a vortex-lattice solve with source-panelled bodies, a sectional viscous coupling, an anchored post-stall section model, and a rotating-frame lattice for the propulsor with a two-way rotor–vane coupling. These methods answer questions one attitude at a time. A flight simulator, a trim routine, or a control allocator instead needs the whole envelope available at once, evaluated in microseconds, in a form that no longer depends on the solver being installed.

DAVE-ML—the Dynamic Aerospace Vehicle Exchange Markup Language, the encoding defined by the American National Standard ANSI/AIAA S-119-2011 [1] and its Annex B reference document [2]—is the standard interchange for exactly this: gridded tables, their breakpoints, the algebraic buildup that assembles them, and, importantly, machine-checkable verification data carried in the same document. This report specifies the Aetherion model in that language.

1.1 Standards conformance

The model is written against the following documents, and departures from any of them are stated where they occur.

The body frame of Section 3— x forward, y toward the starboard wing, z down, with moments positive by the right-hand rule about those axes—is the standard aeronautical body axis system of ANSI/AIAA R-004-1992 [6], itself an adaptation of ISO 1151-1:1988 [7]. The reduced-rate nondimensionalisation of Eq. (14) and the moment coefficient reference lengths follow the same document. Adopting the standard frame is what makes the solver-side conversion of Eq. (1) necessary rather than optional: the solver’s internal axes are not the standard ones, so the boundary between them must be explicit.

Table 4: Governing standards.

Document	Ref.	Governs
ANSI/AIAA S-119-2011	[1]	the exchange standard itself; Annex A standard variable names, Annex B the DAVE-ML reference
DAVE-ML Reference 2.0.1	[2]	element syntax, the <code>DAVEfunc</code> DTD, <code>checkData</code> semantics
ANSI/AIAA R-004-1992	[6]	body-axis definitions, angle and force/moment sign conventions
ISO 1151-1:1988	[7]	flight-dynamics concepts, quantities and symbols; the basis of R-004
XML 1.1, MathML 2.0	[8, 9]	the two open standards DAVE-ML is built upon

One known non-conformance. S-119 Annex A prescribes standard variable names for common flight-dynamics quantities. The variable identifiers in this model (Table 7) were chosen for namespacing—the `aero*`, `prop*`, `coupling*` grouping described below—and have not yet been reconciled against Annex A. Reconciliation is an open item (Section 14); until it is done, a consumer expecting Annex A names must map them, and the file’s variable descriptions carry enough information for that mapping to be mechanical.

The report covers the model’s structure and conventions, the complete set of variables, equations and tables, the provenance of each table, and the declared limits. It does not reproduce the underlying theory, which is documented in `doc/theory.rst` and in the three journal manuscripts the tables draw on [10, 11, 12].

1.2 What is deliberately absent

A tabulated model is a claim about what the generating method knows. Three axes a reader might expect are absent by decision, and their absence is a statement of honesty rather than an omission of work:

- **No Mach axis.** Every solver in the toolkit is incompressible. Mach number is accepted by the section-model interfaces and explicitly ignored; there is no Prandtl–Glauert correction anywhere in the chain. A Mach breakpoint would therefore be a fabricated axis: the generator would produce identical numbers at every breakpoint, and a consumer would silently trust the resulting flat interpolation as physics. Validity $M < 0.3$ is declared in the file header instead.
- **No Reynolds axis.** All airframe sweeps were run at a single flight condition ($V_\infty = 25$ m/s, $Re_n \approx 3 \times 10^5$). The separation behaviour that shapes the post-stall tables was computed at that condition and at no other. The reference condition is declared; an axis is not invented.
- **No disk-incidence axis on the propulsor.** See Section 8.3. The rotor–vane machinery is axisymmetric end to end, so non-axial inflow is not representable without new modelling.

2 Model architecture

2.1 One file, not two

The conventional split places airframe aerodynamics in one DAVE-ML file and propulsion in another. This model does not, for a specific reason: the power-on interaction increments $\Delta C(\alpha, T_c)$ are indexed by an airframe breakpoint (α) and a propulsion-derived state (T_c), and DAVE-ML has no mechanism by which one file may reference another’s variables or breakpoint sets. Split across two files, the interaction tables’ input contract would be enforced only by naming discipline between two documents that version independently.

A single file additionally permits the total-wrench buildup of Section 9—the dimensionalisation of each coefficient group by its own reference quantity and the summation of the three contributions—to live inside the exchanged model as MathML `calculation` elements. This moves the most error-prone step in any consumer integration from the consumer’s code into territory that the file’s own verification data can pin.

The cost of one file is coarser regeneration granularity, and it is avoided by construction: each sweep driver exports its own JSON, and a separate assembler builds the XML from the cached JSONs (Section 12). Regenerating the vane map does not rerun the post-stall map. Variable identifiers are namespaced `aero*`, `prop*`, and `coupling*`, so that a split into separate files—should a consumer ever require it—is a mechanical transformation rather than a redesign.

2.2 Structure at a glance

The file carries, in order: a header declaring provenance, reference condition, and validity envelope; the input variable definitions (Table 7); the constant definitions (Table 8); the derived variables as MathML calculations (Section 5.3); the breakpoint sets (Table 9); the 36 gridded table definitions grouped by namespace (Section 8); the buildup (Section 9); and the `checkData` static shots (Section 11).

3 Reference frames, signs, and the moment reference

This section is normative. Every quantity in the file obeys it, and the generator implements it at exactly two code sites.

3.1 The two frames

The geometry contract states its own frame once and requires the consumer to convert explicitly at ingest. That frame, `aetherion_body_frd`, is x forward, y right, z down—the standard flight-dynamics body frame, and the frame of every quantity in the DAVE-ML file. The solver works internally in x aft, y right, z up. The two are related by a rotation of 180° about y :

$$x_{\text{frd}} = -x_{\text{vlm}}, \quad y_{\text{frd}} = y_{\text{vlm}}, \quad z_{\text{frd}} = -z_{\text{vlm}}. \quad (1)$$

Equation (1) is a *proper* rotation. Two consequences follow, and both matter. First, handedness is preserved, so a rotation axis (a pseudovector) transforms exactly as an ordinary vector does, and the right-hand rule about a converted axis still means what it meant before conversion. Second, forces and moments transform identically under it, giving Table 5.

The generator realises the conversion as

$$F_x = -D_i, \quad F_y = Y, \quad F_z = -L, \quad M_x = -M_x^{\text{vlm}}, \quad M_y = M_y^{\text{vlm}}, \quad M_z = -M_z^{\text{vlm}}, \quad (2)$$

where D_i , Y , L are the solver’s streamwise, lateral and normal force components.

Table 5: Behaviour of each wrench component under the frame conversion of Eq. (1).

Component	Transformation	Physical quantity
C_X, C_Z	sign flips	axial and normal force
C_Y	invariant	side force
C_l, C_n	sign flips	rolling and yawing moment
C_m	invariant	pitching moment

3.2 A recorded failure, and the check that now catches it

The conversion in Eq. (1) applies to the moment reference *point* as much as to the wrench. Omitting it there does not produce an obviously wrong answer; it places the reference point an equal distance on the wrong side of the origin, introducing a spurious moment arm of twice the offset. This occurred once in this codebase: a sweep driver passed the contract’s moment reference point into the solver without the flip, producing a moment-arm error of approximately 0.48 m—about one body length—and inflating $C_{m\alpha}$ by an order of magnitude. The error was invisible to every existing test, because no test compared moments across two reference points.

The model therefore adopts two structural defences. First, the generator performs the conversion at exactly two sites—reference-point ingest and wrench export, Eq. (2)—and nowhere else, so that there is one place to audit rather than a scattering of sign flips. Second, the file carries a *moment-transfer static shot*: the same flight condition exported about the contract’s reference point and about a point displaced +0.10 m in x_{frd} . The difference in C_m must equal the hand-derivable transfer arm,

$$\Delta C_m = \frac{\Delta x F_z - \Delta z F_x}{\bar{q} S \bar{c}}, \quad (3)$$

evaluated at $\Delta x = 0.10$ m, $\Delta z = 0$. A wrong flip fails this shot by 2×0.2423 m of arm, roughly two reference chords, which cannot be mistaken for numerical noise.

3.3 Moment reference point

All moments in the file are taken about the contract’s stated moment reference point,

$$(x, y, z)_{\text{mrp}} = (-0.2423, 0, 0) \text{ m}, \quad \text{contract frame (FRD)}, \quad (4)$$

declared twice in the document: as prose in the file header, and as three constant variable definitions `XmrpM`, `YmrpM`, `ZmrpM`. The redundancy is deliberate—a consumer transferring the model to its own centre of gravity needs the point as data, not as a comment, and the transfer is then Eq. (3) generalised to all three axes:

$$M'_x = M_x - \Delta y F_z + \Delta z F_y, \quad (5)$$

$$M'_y = M_y - \Delta z F_x + \Delta x F_z, \quad (6)$$

$$M'_z = M_z - \Delta x F_y + \Delta y F_x, \quad (7)$$

with $\Delta = \mathbf{r}_{\text{new}} - \mathbf{r}_{\text{mrp}}$.

3.4 Deflection sign convention

Positive deflection of any control surface is the right-hand rule about that surface’s hinge-axis vector *as stated in the contract’s FRD frame*. This is the entire normative statement. English glosses (“positive is trailing edge down”) are excluded from the file’s normative text, because such a description must be re-derived in each frame and a re-derivation that goes wrong inverts half the controls silently. The numeric meaning of each sign is instead pinned by a generated static shot, and each control’s variable definition cites the shot that defines it.

4 Nondimensionalisation

Three normalisation groups appear in the model, and mixing them is the most likely consumer error—which is why the buildup of Section 9 is carried inside the file.

Airframe coefficients use free-stream dynamic pressure and wing reference geometry:

$$C_X = \frac{F_x^{\text{aero}}}{\bar{q}S}, \quad C_Y = \frac{F_y^{\text{aero}}}{\bar{q}S}, \quad C_Z = \frac{F_z^{\text{aero}}}{\bar{q}S}, \quad C_l = \frac{M_x^{\text{aero}}}{\bar{q}Sb}, \quad C_m = \frac{M_y^{\text{aero}}}{\bar{q}S\bar{c}}, \quad C_n = \frac{M_z^{\text{aero}}}{\bar{q}Sb}, \quad (8)$$

with

$$\bar{q} = \frac{1}{2}\rho V_\infty^2. \quad (9)$$

Propulsor coefficients use the propeller convention, which remains finite at zero airspeed and is therefore the only viable choice for a tail-sitter that must be modelled in hover:

$$C_T = \frac{T}{\rho n^2 D^4}, \quad C_Q = \frac{Q}{\rho n^2 D^5}, \quad (10)$$

with n in revolutions per second. The declared validity excludes $n \rightarrow 0$, where this group degenerates; windmilling and low-rotor-speed descent are outside the model’s envelope and are identified as future work.

4.1 Drag accounting

Drag deserves its own statement, because the body-axis force tables are routinely misread as carrying induced drag alone, and because one of the three physical contributions is not computed anywhere in the toolkit.

Table 6: The three drag contributions, where each lives, and its status.

Contribution	Carried by	Status
Induced	<code>aeroCX</code> , <code>aeroCZ</code>	computed (near-field Kutta–Joukowski; cross-checked against a Trefftz-plane integral)
Wing section profile	the same tables	computed—the section model’s c_d is integrated per strip and folded into the same force vector
Body and duct parasite	<code>aeroCDO</code>	computed by a component buildup plus a crossflow branch, Eq. (11)

The first point is that the airframe force tables are *not* induced-drag tables. The Level-2 coupled solve accumulates the circulatory (Kutta–Joukowski) force and the sectional profile force into one total per strip, so the exported body-axis coefficients already contain the wing’s viscous drag as the section model computes it. The attached-flow study [10] reports induced drag alone, both near-field and by a Trefftz-plane integral; the post-stall study [11] reports induced plus profile. Neither reports a complete drag polar.

The second point is the contribution the solves cannot give. A potential-flow method cannot produce the parasite drag of a closed body: source-panelled bodies carry zero net force in uniform flow to machine precision, which is d’Alembert’s paradox behaving correctly rather than a defect.

That contribution must come from a component buildup—skin friction with form and interference factors, in the manner of Raymer and Hoerner [13, 14]—driven by wetted areas assembled from the geometry contract.

Two consequences shape the model, and both are easy to get wrong:

1. **A whole-airframe C_{D_0} must never be added on top of these tables.** The wing’s skin friction is already inside the section c_d . The parasite term admitted to this model is *body and duct wetted area only*; adding a conventional airframe buildup would double-count the wing.
2. **Parasite drag cannot be a constant here.** The tables span α to 90° . A slender body broadside to the flow carries a crossflow drag of order unity on its projected area—comparable to the entire tabulated normal force—and that contribution varies strongly with attitude. A single constant C_{D_0} , the conventional choice for a cruise-envelope model, would underpredict axial force in precisely the deep-stall regime this model exists to cover.

Parasite drag is therefore tabulated as `aeroCD0`(α): a component buildup for the friction branch plus a slender-body crossflow term,

$$C_{D_0}(\alpha) = C_{D_0, \text{friction}} + \eta C_{d_c} \frac{S_{\text{plan}}}{S} |\sin^3 \alpha|, \quad (11)$$

following Allen and Perkins and Jorgensen [15, 16], with S_{plan} the body’s side-projected area, $C_{d_c} = 1.2$ the subcritical circular-cylinder crossflow drag coefficient, and $\eta = 0.65$ the finite-length proportionality factor at this body’s fineness ratio of 5. The measured values are in Section 10.1.

In the assembled model the parasite term is resolved into body axes along the velocity direction and added to the force buildup of Section 9, so that the total drag

$$C_D = \underbrace{C_{D_i} + C_{d, \text{wing}}}_{\text{inside } \text{aeroCX}, \text{aeroCY}, \text{aeroCZ}} + \underbrace{\text{aeroCD0}(\alpha)}_{\text{body + duct}} \quad (12)$$

is recovered from the assembled body-axis components by the projection onto the free-stream direction of Eq. (17),

$$C_D = -(C_X \cos \alpha \cos \beta + C_Y \sin \beta + C_Z \sin \alpha \cos \beta), \quad (13)$$

with no further term: once the buildup has run, every drag contribution is already inside the body-axis components, and adding `aeroCD0` to Eq. (13) would count it twice.

Rate derivatives use the conventional reduced-rate nondimensionalisation,

$$\hat{p} = \frac{pb}{2V_\infty}, \quad \hat{q} = \frac{q\bar{c}}{2V_\infty}, \quad \hat{r} = \frac{rb}{2V_\infty}, \quad (14)$$

so that $C_{l_p} = \partial C_l / \partial \hat{p}$ and so forth. This convention is worth stating explicitly because its reciprocal was once implemented in this codebase: the reduced-rate fields multiplied by $b/2V_\infty$ where Eq. (14) requires division by that factor—an error of $(2V_\infty/b)^2 \approx 2000$ at these numbers, which reported $C_{l_p} = -2 \times 10^{-4}$ for a wing whose textbook value is -0.45 . With the convention corrected the solver returns -0.452 , and that value is now pinned both by a unit test and by a static shot in this file.

5 Variables

5.1 Inputs

Table 7: Input variable definitions.

varID	Units	Meaning
alphaDeg	deg	angle of attack α
betaDeg	deg	sideslip angle β
trueAirspeedMps	m/s	true airspeed V_∞
airDensityKgpm3	kg/m ³	air density ρ
rollRateRadps	rad/s	body roll rate p
pitchRateRadps	rad/s	body pitch rate q
yawRateRadps	rad/s	body yaw rate r
propSpeedRevps	rev/s	rotor speed n
aileronDeg	deg	antisymmetric aileron command δ_a , right surface; the left surface deflects to $-\delta_a$
vanePitchDeg	deg	vane pitch mode command δ_P
vaneYawDeg	deg	vane yaw mode command δ_Y
vaneRollDeg	deg	vane roll mode command δ_R

5.2 Constants

Values are transcribed by the assembler from the geometry contract, never typed by hand.

Table 8: Constant variable definitions (contract 1.8.0 values).

varID	Value	Units	Meaning
WingAreaM2	0.1883	m ²	gross planform reference area S
WingSpanM	1.0629	m	reference span b
WingChordM	0.1771	m	reference chord $\bar{c}$
DiskDiameterM	0.2030	m	rotor diameter D
XmrpM	-0.2423	m	moment reference point, FRD
YmrpM	0	m	
ZmrpM	0	m	

Parasite drag is deliberately *not* among the constants; it is a table, for the reason developed in Section 4.1.

The reference planform is rectangular and unswept ($b = 1.0629$ m, $\bar{c} = 0.1771$ m), giving aspect ratio $AR = 6.0$ —a number that recurs in the post-stall anchors of Section 8.1. The rotor is a three-bladed fan of diameter 0.2030 m within a duct of bore diameter 0.2090 m and chord 0.09135 m, centred at $x_{\text{frd}} = -0.46787$ m.

5.3 Derived variables

These are MathML `calculation` elements inside the file, evaluated before any table lookup.

$$\bar{q} = \frac{1}{2} \rho V_\infty^2 \quad \text{dynamic pressure} \quad (15)$$

$$J = \frac{V_\infty}{nD} \quad \text{advance ratio} \quad (16)$$

$$\hat{u} = (\cos \alpha \cos \beta, \sin \beta, \sin \alpha \cos \beta) \quad \text{free-stream unit vector, FRD} \quad (17)$$

$$\alpha_{\text{disk}} = \arccos(\cos \alpha \cos \beta) \quad \text{disk incidence (validity monitor)} \quad (18)$$

$$\phi_w = \text{atan2}(\sin \beta, \sin \alpha \cos \beta) \quad \text{crossflow azimuth (diagnostic)} \quad (19)$$

$$\sigma = \arcsin(\sin \alpha \cos \beta) \quad \text{total inclination} \quad (20)$$

$$\hat{p} = \frac{pb}{2V_\infty}, \quad \hat{q} = \frac{q\bar{c}}{2V_\infty}, \quad \hat{r} = \frac{rb}{2V_\infty} \quad \text{reduced rates} \quad (21)$$

$$T_c = \frac{T}{\bar{q}S} \quad \text{interaction index} \quad (22)$$

Three of these warrant comment.

Equations (18) and (19) are monitors, not table indices. With the rotor axis along body $+x$, the angle between the free stream and the axis follows from the first component of Eq. (17), and ϕ_w is the azimuth of the crossflow about that axis, zero in the pitch plane. Neither indexes a table, for the reason given in Section 8.3: the propulsor model is axial-inflow only. They are computed and exported precisely so that a consumer can *detect* when it has left the model's validity envelope, which is a more useful service than an axis the solver cannot honestly fill.

Equation (20) is the collapse variable of the post-stall study: beyond separation the loading depends on attitude almost entirely through σ , with cross- β spread within 13% from $\alpha \approx 6^\circ$ through 75° . It is exported as a diagnostic and is the natural coordinate in which to inspect the post-stall tables for consistency.

Equation (22) closes no algebraic loop. The propulsor wrench does not depend on T_c , so thrust is available from the `prop*` tables before the `coupling*` tables are evaluated. The evaluation order is fixed: derived scalars, propulsor tables, T_c , airframe tables, interaction tables, buildup.

6 The vane command space

6.1 Why not four independent angles

The duct-exit cruciform has four all-moving vanes, and the contract states each one's hinge axis independently. Tabulating four independent angles would require $7^4 = 2401$ combinations before the advance-ratio axis is considered—infeasible, and structurally wrong, because a flight controller does not command vanes individually. It commands moments.

6.2 Mode shapes from the geometry

All four hinge axes in the contract point radially *outward*: $[0, 0, 1]$ (bottom, since z is down), $[0, -1, 0]$ (left), $[0, 0, -1]$ (top), and $[0, 1, 0]$ (right). Applying an equal right-hand-rule rotation about four outward radial axes tilts every plate with the same handedness as seen from outside the duct, so all four tangential forces circulate the same way about the x axis: the net force cancels by symmetry and a pure rolling couple survives. Differential deflection of one opposed

pair instead redirects the jet transversely, producing a net force and the associated moment. Hence the mixing

$$\begin{pmatrix} \delta_b \\ \delta_l \\ \delta_t \\ \delta_r \end{pmatrix} = \begin{pmatrix} 0 & 1 & 1 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} \delta_P \\ \delta_Y \\ \delta_R \end{pmatrix}, \quad (23)$$

in which the third column is the common mode (roll), the first is the y -pair differential (pitch, redirecting the jet in z), and the second is the z -pair differential (yaw, redirecting it in y). The *numeric* signs in Eq. (23)—which member of each pair receives the positive angle—are a convention, and the file pins them by static shot rather than by argument, for the reason given in Section 3.

6.3 Three properties the tables must respect

Each mode carries a full wrench. Rotor swirl means the two vanes of a differential pair meet the jet at different local incidence, so a pitch command produces measurable yaw and roll components. These cross-terms are recorded rather than assumed zero—see the generated Table 14, where they are visible. Each mode table also carries ΔC_Q , the back-pressure on the rotor, because the rotor–vane coupling is two-way: vane blockage and upwash enter the rotor’s boundary condition, and the rotor’s loading shifts measurably when the vanes deflect.

Deflection is not antisymmetric in δ . The vane sits in a jet whose dynamic pressure and swirl are set by the rotor, not by the free stream, so the loads at $+\delta$ and $-\delta$ are not mirror images. The tables therefore carry both signs explicitly; no mirroring is applied.

Superposition was measured, and it partly fails. The obvious buildup—tabulate three modes, add them—was tested rather than assumed, and the test changed the model. The result is Section 10.2: pitch and yaw superpose to better than 0.1%, but any pair involving roll is wrong by 6 to 33%. The mechanism is simple once seen, and it is the reason the test was worth running.

6.4 Travel limits

The contract states mechanical stops of $\pm 20^\circ$ and a soft limit of $\pm 15^\circ$. The tables are generated over $\pm 15^\circ$, and an allocator consuming this model should saturate the *sum* of the three mode commands at each individual vane—i.e. each row of Eq. (23)—against the soft limit. That limit exists precisely because a vane can be driven to its mechanical stop but should not be in normal operation.

7 Breakpoint sets

The α grid is deliberately non-uniform: 2° spacing through the stall break, where the anchored section model produces $C_{L\max}$ at $\alpha = 18^\circ$ and the loads change rapidly, coarsening to $5\text{--}10^\circ$ in the plate regime where the loading varies slowly with attitude. The β grid exploits the unpowered airframe’s mirror symmetry,

$$C_Y, C_l, C_n \text{ odd in } \beta; \quad C_X, C_Z, C_m \text{ even in } \beta, \quad (24)$$

a property confirmed numerically in the source sweeps rather than merely assumed. The T_c grid is logarithmically spaced because the induced velocity ratio that drives the interaction varies as $\sqrt{T_c}$ in momentum theory, so equal ratios of T_c correspond to roughly equal increments of effect.

Table 9: Breakpoint sets. Angles in degrees.

bpID	Count	Values
alphaBp	25	-4, -2, 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 30, 35, 40, 45, 50, 60, 70, 80, 90
betaBp	13	0, ± 5 , ± 10 , ± 15 , ± 20 , ± 25 , ± 30 (generated at $\beta \geq 0$, mirrored)
aileronBp	7	-20, -10, -5, 0, 5, 10, 20
jBp	11	0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0
vaneBp	7	-15, -10, -5, 0, 5, 10, 15
tcBp	6	0, 0.5, 1, 2, 4, 8

8 Table inventory and provenance

Every table is listed. Each group states which solver path generated it, so that any number in the model can be traced to a run.

8.1 aero*: airframe, power-off

Source: the Level-2 coupled solve—vortex lattice with source-panelled body and duct, sectional viscous feedback, and the anchored post-stall section model (Kirchhoff attenuation driven by a computed separation point, Viterna–Corrigan deep stall [17] with aspect-ratio-aware $C_{d\max}$, and Rayleigh free-streamline centre of pressure).

Table 10: Airframe tables (21).

varID	Axes	Quantity	Notes
aeroCX	$\alpha \times \beta$	C_X	baseline, controls neutral, power-off
aeroCY	$\alpha \times \beta$	C_Y	
aeroCZ	$\alpha \times \beta$	C_Z	
aeroCl	$\alpha \times \beta$	C_l	
aeroCm	$\alpha \times \beta$	C_m	
aeroCn	$\alpha \times \beta$	C_n	
aeroCZq	α	$\partial C_Z / \partial \hat{q}$	from CL_q_nd, sign per Table 5
aeroCmq	α	$\partial C_m / \partial \hat{q}$	from Cm_q_nd
aeroClp	α	$\partial C_l / \partial \hat{p}$	from Croll_p_nd
aeroCnp	α	$\partial C_n / \partial \hat{p}$	from Cn_p_nd
aeroClr	α	$\partial C_l / \partial \hat{r}$	from Croll_r_nd
aeroCnr	α	$\partial C_n / \partial \hat{r}$	from Cn_r_nd
aeroCYp	α	$\partial C_Y / \partial \hat{p}$	converted by the assembler from the per-rad/s field
aeroCYr	α	$\partial C_Y / \partial \hat{r}$	likewise
aeroDCX	$\alpha \times \delta_a$	ΔC_X	aileron increment from baseline
aeroDCY	$\alpha \times \delta_a$	ΔC_Y	retained though symmetry expects ≈ 0
aeroDCZ	$\alpha \times \delta_a$	ΔC_Z	
aeroDCI	$\alpha \times \delta_a$	ΔC_l	the primary roll authority

Table 10, continued

varID	Axes	Quantity	Notes
aeroDCm	$\alpha \times \delta_a$	ΔC_m	
aeroDCn	$\alpha \times \delta_a$	ΔC_n	adverse yaw
aeroCD0	α	C_{D0}	body and duct parasite <i>only</i> , never the wing (Section 4.1)

All rate-derivative tables are generated at $\beta = 0$ and carry the taper of Section 8.1. The eight comprise the full set the solver produces in reduced-rate form; $\partial C_X / \partial \hat{q}$ is omitted because the underlying induced-drag derivative reflects induced drag only and is not a trustworthy axial-force rate term.

The overlap decision. Two solver paths cover $\alpha \in [6^\circ, 16^\circ]$: the attached-flow matrix of Part I [10] and the coupled post-stall solve of Part II [11]. Tabulating one range from each would introduce a seam in the middle of the most flight-critical region. The model takes the Part II coupled solve as its single source across the whole α range, because that solve reduces to the attached solution below separation by construction; the Part I matrix is retained as a cross-check in the verification data rather than as table content. There is therefore exactly one solver path behind every airframe number, and no blend to justify.

Post-stall values are cycle means. Beyond separation the coupled fixed point does not converge to a steady state; it enters a limit cycle, and the exported value is the cycle mean, flagged unconverged in the source JSON with its residual. Independent iteration paths agree on these means to four or more digits, so they are reproducible quantities—but a consumer flying a smooth interpolation through this region is modelling a regime that physically buffets. The tables therefore carry DAVE-ML uncertainty bounds derived from the cycle RMS, which is the mechanism the standard provides for exactly this situation.

Anchors worth stating. The anchored model produces $C_{L\max} = 1.76$ at $\alpha = 18^\circ$, emergent from the computed separation tables rather than fitted; $C_N(90^\circ) = 1.52$ against the Viterna anchor

$$C_{d\max} = 1.11 + 0.018 AR = 1.218 \quad (AR = 6), \quad (25)$$

and a centre of pressure that walks from the attached quarter chord to $x_{cp} = 0.52\bar{c}$, landing on Rayleigh’s free-streamline mid-chord—a metric the model was never fitted to. $C_{L\max}$ is an *upper bound*: bubble bursting is not modelled, so the usable incidence limit may be lower than the tabulated stall.

Rate derivatives past stall. The derivatives are computed by finite difference on the attached solve and have no meaning once the flow separates. Both available treatments—holding the last attached value, or zeroing discontinuously—are wrong; the model applies a linear taper

$$C_{(\cdot)}(\alpha) \longrightarrow C_{(\cdot)}(\alpha) \cdot \max\left(0, \min\left(1, \frac{40^\circ - \alpha}{20^\circ}\right)\right), \quad (26)$$

which is the conservative choice in the sense that it does not grant a simulator attached-flow damping in deep stall. This is a declared assumption, not a computed result.

8.2 prop*: rotor, duct, and vanes

Source: the two-way rotor–vane coupled solve—a block Gauss–Seidel alternation of the rotating-frame rotor-plus-duct solve and the static-frame vane solve, with an angular-momentum budget on the swirl. This is the method validated in the SciTech manuscript [12]. Thrust is recovered as $-D_i$ and shaft torque as $-M_x$ of the rotating-frame solve, both then converted by Eq. (2).

Table 11: Propulsor tables (9). The vane tables are *per-vane*, not per-mode: one vane’s response to its own deflection, serving all four positions by the cruciform’s rotational symmetry. See Section 10.2 for the measurement that forced this structure.

varID	Axes	Quantity	Notes
propCT	J	C_T	vanes neutral; includes duct and vane axial force
propCQ	J	C_Q	shaft torque
propDCXvane	$J \times \delta_i$	ΔC_X	one vane’s own increment, normalised per Eq. (10)
propDCYvane	$J \times \delta_i$	ΔC_Y	
propDCZvane	$J \times \delta_i$	ΔC_Z	
propDClvane	$J \times \delta_i$	ΔC_l	
propDCmvane	$J \times \delta_i$	ΔC_m	
propDCnvane	$J \times \delta_i$	ΔC_n	
propDCQvane	$J \times \delta_i$	ΔC_Q	two-way back-pressure on the shaft

8.3 Axial inflow only: a declared structural limit

The propulsor tables carry no disk-incidence axis. This is not an economy; it is a statement about what the method can represent.

The rotor–vane coupling is built on axisymmetry at every stage. The slipstream is reconstructed as radial bands of axial and swirl velocity; the vanes’ influence on the rotor enters as an azimuthally averaged induced field; the self-consistent wake pitch is set by a radial inflow distribution $v_i(r)$. Each of these assumes the loading is independent of azimuth. Under non-axial inflow it is not: a blade advancing into the crossflow meets a different relative velocity than one retreating, and the resulting once-per-revolution loading produces the very in-plane force and hub moment that a disk-incidence table would be created to carry.

Generating an α_{disk} sweep from this solver would therefore produce numbers that vary—the free-stream projection changes with incidence—while omitting the physical mechanism the axis exists to capture. That is worse than an absent axis: it is a plausible artifact. The model declares axial inflow, exports α_{disk} as a validity monitor (Eq. 18), and records non-axial propulsor modelling as identified future work.

For a tail-sitter this limitation is real and bounded: the model is exact in hover and in axial climb, and it degrades through the transition corridor, where the free stream meets the disk at increasing incidence. The interaction increments of Section 8.4 are subject to the same bound.

8.4 coupling*: fan-on-airframe increments

This block is specified but not yet generable.

Table 12: Interaction tables (6). Blocked pending an upstream-induction model.

varID	Axes	Quantity	Notes
couplingDCX	$\alpha \times T_c$	ΔC_X	$\beta = 0$; valid $V_\infty \geq 10$ m/s
couplingDCY	$\alpha \times T_c$	ΔC_Y	
couplingDCZ	$\alpha \times T_c$	ΔC_Z	
			the separation delay appears here
couplingDCI	$\alpha \times T_c$	ΔC_l	
couplingDCm	$\alpha \times T_c$	ΔC_m	
couplingDCn	$\alpha \times T_c$	ΔC_n	

The mechanism is specific to this configuration and worth stating precisely, because it is frequently misidentified. The ducted fan sits *aft* of the wing: the duct leading edge is 0.21 chords behind the wing trailing edge. The fan therefore does not blow the wing. What it does is induce a favourable pressure gradient *upstream* of itself, over the wing, delaying separation—and it does so over precisely the span that sheds first (duct outer radius to semi-span ratio 0.21, against a worst separation station at $|2y/b| \approx 0.15\text{--}0.23$).

The existing slipstream model cannot compute this: it is a momentum-theory wake and returns identically zero upstream of the disk by construction. Applied here it would report exactly zero interaction—an artifact indistinguishable from a physical null result. A semi-infinite vortex cylinder, which has a closed-form upstream field, is the identified prerequisite, and the interaction tables are blocked on it. The file’s structure reserves their place so that adding them later does not perturb anything else.

9 The buildup

The file assembles the total wrench in MathML, so that a consumer receives forces and moments rather than a set of coefficients it must combine correctly. Airframe coefficients accumulate baseline, control, interaction and rate contributions:

$$C_X = \text{aeroCX}(\alpha, \beta) + \text{aeroDCX}(\alpha, \delta_a) + \text{couplingDCX}(\alpha, T_c) - \text{aeroCD0}(\alpha) \cos \alpha \cos \beta \quad (27)$$

$$C_Y = \text{aeroCY}(\alpha, \beta) + \text{aeroDCY}(\alpha, \delta_a) + \text{couplingDCY}(\alpha, T_c) + \text{aeroCYp} \hat{p} + \text{aeroCYr} \hat{r} - \text{aeroCD0}(\alpha) \sin \beta \quad (28)$$

$$C_Z = \text{aeroCZ}(\alpha, \beta) + \text{aeroDCZ}(\alpha, \delta_a) + \text{couplingDCZ}(\alpha, T_c) + \text{aeroCZq} \hat{q} - \text{aeroCD0}(\alpha) \sin \alpha \cos \beta \quad (29)$$

$$C_l = \text{aeroCl}(\alpha, \beta) + \text{aeroDCI}(\alpha, \delta_a) + \text{couplingDCI}(\alpha, T_c) + \text{aeroClp} \hat{p} + \text{aeroClr} \hat{r} \quad (30)$$

$$C_m = \text{aeroCm}(\alpha, \beta) + \text{aeroDCm}(\alpha, \delta_a) + \text{couplingDCm}(\alpha, T_c) + \text{aeroCmq} \hat{q} \quad (31)$$

$$C_n = \text{aeroCn}(\alpha, \beta) + \text{aeroDCn}(\alpha, \delta_a) + \text{couplingDCn}(\alpha, T_c) + \text{aeroCnp} \hat{p} + \text{aeroCnr} \hat{r} \quad (32)$$

Propulsor coefficients accumulate the baseline and the vane contribution:

$$C_X^{\text{prop}} = \text{propCT}(J) + \Delta C_X^{\text{vane}}, \quad (33)$$

$$C_k^{\text{prop}} = \Delta C_k^{\text{vane}}, \quad k \in \{Y, Z, l, m, n\}, \quad (34)$$

$$C_Q^{\text{prop}} = \text{propCQ}(J) + \Delta C_Q^{\text{vane}}. \quad (35)$$

The vane contribution is *not* a sum over the three command modes. That structure was tested and rejected on measurement (Section 10.2). It is instead a sum over the four physical vanes, each evaluated at its own total commanded angle from Eq. (23):

$$\Delta C_k^{\text{vane}} = \sum_{i \in \{\text{b}, \text{l}, \text{t}, \text{r}\}} \text{propDCkvane}(J, \delta_i), \quad \delta_i = [\mathbf{M}(\delta_P, \delta_Y, \delta_R)^\top]_i, \quad (36)$$

with $\mathbf{M}$ the mixing matrix of Eq. (23). A single per-vane table—one vane’s response to its own deflection, the other three neutral—serves all four positions by the cruciform’s rotational symmetry, with the wrench components permuted accordingly. This replaces 21 mode tables with 7 per-vane tables and, unlike the mode sum, respects the measured physics.

Dimensionalisation then uses each group’s own reference quantities, and the two groups sum in body axes:

$$F_x = \bar{q}S C_X + \rho n^2 D^4 C_X^{\text{prop}} \quad (37)$$

$$F_y = \bar{q}S C_Y + \rho n^2 D^4 C_Y^{\text{prop}} \quad (38)$$

$$F_z = \bar{q}S C_Z + \rho n^2 D^4 C_Z^{\text{prop}} \quad (39)$$

$$M_x = \bar{q}S b C_l + \rho n^2 D^5 C_l^{\text{prop}} \quad (40)$$

$$M_y = \bar{q}S \bar{c} C_m + \rho n^2 D^5 C_m^{\text{prop}} \quad (41)$$

$$M_z = \bar{q}S b C_n + \rho n^2 D^5 C_n^{\text{prop}} \quad (42)$$

Both groups’ moments are already referred to the same point—the generator exports the propulsor wrench about the contract’s moment reference point, not about the disk centre—so no transfer term appears in the sum. Thrust and shaft torque are exported alongside as

$$T = \rho n^2 D^4 C_X^{\text{prop}}, \quad Q = \rho n^2 D^5 C_Q^{\text{prop}}, \quad (43)$$

because T feeds Eq. (22) and Q is what a powertrain model needs.

10 First generated results

The propulsor generator has been run against the 1.8.0 geometry contract at the contract’s reference rotor speed. The tables below are reproduced verbatim from the generated JSON by `make-tables.py`—they are included as evidence that the pipeline closes end to end, not as a validated performance prediction, which awaits the anchor data identified in [12].

Table 13: Generated propulsor baseline: the contract’s ducted fan at $n = 203.5$ rev/s ($\Omega = 1279$ rad/s), $D = 0.2030$ m, 3 blades, $\rho = 1.225$ kg/m³, vanes neutral. Forces and moments in FRD about the contract’s moment reference point. Q is the shaft torque, i.e. the jet’s angular-momentum flux.

J	V_∞ [m/s]	T [N]	Q [N m]	C_T	C_Q	M_x [N m]	conv.
0.00	0.00	15.121	0.1632	0.1755	0.00933	-0.0113	yes
0.10	4.13	13.181	0.1729	0.1530	0.00988	-0.0132	yes
0.20	8.26	11.398	0.1754	0.1323	0.01003	-0.0165	yes
0.30	12.39	10.058	0.1787	0.1167	0.01022	-0.0192	yes
0.40	16.53	8.687	0.1767	0.1008	0.01010	-0.0208	yes
0.50	20.66	7.103	0.1643	0.0824	0.00939	-0.0203	yes
0.60	24.79	5.223	0.1390	0.0606	0.00795	-0.0190	yes
0.70	28.92	3.153	0.1002	0.0366	0.00573	-0.0135	yes
0.80	33.05	0.874	0.0453	0.0101	0.00259	-0.0052	yes
0.90	37.18	-1.201	-0.0217	-0.0139	-0.00124	0.2156	yes
1.00	41.31	-1.834	-0.0971	-0.0213	-0.00555	0.7446	yes

The behaviour is qualitatively correct for a ducted fan: thrust falls monotonically with advance ratio as the blades unload, and the cruciform’s lateral force and moment components vanish at

neutral deflection, as Eq. (23) with $\delta_P = \delta_Y = \delta_R = 0$ requires. The swirl-recovery vanes return the large majority of the shaft torque, leaving a small residual net rolling moment—which is the design intent of a swirl-recovery cruciform, and the reason the aircraft’s rolling moment cannot be read off the rotor alone.

The two highest advance ratios are outside the envelope, and show it. Thrust crosses zero near $J \approx 0.85$ and both C_T and C_Q are negative at $J = 0.9$ and 1.0 : the rotor has entered the windmilling regime, which Section 4 declares outside the model’s validity. The residual rolling moment simultaneously jumps by more than an order of magnitude, from -0.019 to $+0.745$ N m, because the swirl the vanes were sized to recover has reversed while the vanes have not. These rows are retained in the generated map rather than deleted—they are the measured location of the envelope boundary, and knowing where the model stops behaving is worth more than a table that ends without explanation—but the assembler truncates the `jBp` axis below the thrust reversal, and a consumer must not interpolate into that corner. Modelling windmilling properly requires the inflow-angle parameterisation noted in Section 4, not an extension of this grid.

10.1 Parasite drag, and the case against a constant

Two results are worth drawing out. First, the body and the duct contribute almost equally to the friction branch (0.00521 against 0.00507), which is not obvious in advance: the duct’s wetted area is smaller, but its chord is short enough that its Reynolds number is a factor of five lower and its skin-friction coefficient correspondingly higher. A buildup that treated the duct as a minor appendage would lose half the friction drag.

Second, and decisively for the model’s structure, the crossflow branch reaches 0.185 at 90° —**17.5 times** the friction total. Had parasite drag been carried as the single constant that a cruise-envelope model would naturally use, the tabulation would have omitted 95% of the parasite drag at the attitude this model exists to describe. Against the configuration’s $C_N(90^\circ) = 1.52$ the parasite branch is a 13% addition: not dominant, but far outside the tolerance any other number here is quoted to. This is the measurement that converted `aeroCD0` from a constant into a table.

10.2 Vane-mode superposition: measured, and partly false

The natural buildup for a three-mode control system is to tabulate each mode and add the increments, and that is what this model originally specified. It was tested by solving each simultaneous pair of commands directly and comparing against the sum of the corresponding single-mode increments, all taken from the neutral baseline at matched advance ratio. Each mode was commanded at 7.5° so that the per-vane sum—which is what the mechanism actually sees—stays within the $\pm 15^\circ$ soft limit.

The result splits cleanly in two. **Pitch and yaw superpose essentially exactly**, to within 0.04 – 0.09% across every advance ratio and both signs. **Any pair involving roll does not**, with errors from 6% to 33% , worst in hover and at positive commands.

The mechanism follows from the mixing matrix, Eq. (23). Pitch acts on the y -pair and yaw on the z -pair: *disjoint* sets of vanes. Commanding both leaves every vane at the same angle it would have held under one command alone, so the loads add because nothing about any individual vane has changed. Roll is the common mode and deflects *all four*—including the two a pitch or yaw command is already deflecting. Under roll-plus-pitch the starboard vane sits at $\delta_R + \delta_P = 15^\circ$, whereas the two single-mode cases each placed it at 7.5° . A vane’s load is not linear in its own deflection, so the increments cannot add.

Two conclusions follow, and the second is the useful one:

1. **The mode-sum buildup is invalid** and was removed from Section 9. Retaining it would have given a control allocator a **33%** error in hover—precisely the flight condition in which a tail-sitter depends on vane authority most.

Table 14: Generated vane-mode increments at the $+15^\circ$ soft limit, taken from the neutral baseline at matched advance ratio. Each mode carries a full wrench: the off-axis components are the rotor swirl’s signature, not numerical noise. ΔQ is the back-pressure on the shaft, which exists because the rotor–vane coupling is two-way.

Mode	J	ΔF_x [N]	ΔF_y [N]	ΔF_z [N]	ΔM_x [N m]	ΔM_y [N m]	ΔM_z [N m]	ΔQ [N m]
Pitch	0.00	-0.130	0.001	-1.085	0.0179	-0.3181	-0.0019	0
	0.10	-0.142	0.001	-1.220	0.0191	-0.3580	-0.0019	0
	0.20	-0.183	0	-1.513	0.0195	-0.4444	-0.0015	0
	0.30	-0.248	0	-1.910	0.0196	-0.5613	-0.0011	0
	0.40	-0.325	0	-2.361	0.0191	-0.6941	-0.0007	0
	0.50	-0.411	0	-2.848	0.0177	-0.8377	-0.0005	0
	0.60	-0.534	0	-3.578	0.0147	-1.0525	0	-0.0001
	0.70	-0.636	0	-4.180	0.0107	-1.2300	0	-0.0002
	0.80	-0.751	0	-4.900	0.0050	-1.4420	-0.0002	-0.0004
	0.90	-0.869	0	-5.854	-0.0181	-1.7220	-0.0018	-0.0006
	1.00	-0.794	-0.003	-6.233	-0.0820	-1.8282	0.0067	-0.0009
Roll	0.00	-0.571	0.009	-0.001	-0.1333	-0.0005	-0.0025	-0.0023
	0.10	-0.447	0.003	-0.002	-0.1508	-0.0007	-0.0007	-0.0014
	0.20	-0.277	-0.006	-0.002	-0.1911	-0.0006	0.0017	-0.0012
	0.30	-0.364	-0.003	-0.003	-0.2474	-0.0008	0.0008	-0.0004
	0.40	-0.493	-0.002	-0.003	-0.3134	-0.0008	0.0007	0.0008
	0.50	-0.637	-0.002	-0.001	-0.3890	-0.0003	0.0005	0.0023
	0.60	-1.066	-0.003	0	-0.5127	0	0.0008	-0.0047
	0.70	-1.097	-0.003	0	-0.5927	-0.0001	0.0009	0.0035
	0.80	-1.431	-0.002	0	-0.7054	-0.0001	0.0005	0.0011
	0.90	-1.695	0.001	0.001	-0.8664	0.0002	-0.0003	-0.0044
	1.00	-1.630	0.003	0.004	-1.0511	0.0012	-0.0010	-0.0077
Yaw	0.00	-0.066	1.075	0.001	0.0178	0.0019	-0.3152	0.0003
	0.10	-0.140	1.220	0.001	0.0193	0.0019	-0.3578	0
	0.20	-0.183	1.513	0	0.0201	0.0016	-0.4442	0
	0.30	-0.248	1.910	0	0.0200	0.0012	-0.5612	0
	0.40	-0.325	2.361	0	0.0195	0.0008	-0.6940	0
	0.50	-0.411	2.848	0	0.0179	0.0005	-0.8375	0
	0.60	-0.534	3.577	0	0.0148	0	-1.0524	-0.0001
	0.70	-0.636	4.179	0	0.0109	0	-1.2297	-0.0003
	0.80	-0.751	4.899	0	0.0051	0.0002	-1.4418	-0.0004
	0.90	-0.869	5.854	0	-0.0182	0.0018	-1.7220	-0.0006
	1.00	-0.793	6.233	-0.003	-0.0826	-0.0068	-1.8280	-0.0009

Table 15: Parasite drag of the body and duct, referred to the wing reference area $S = 0.1883 \text{ m}^2$ at $V_\infty = 25 \text{ m/s}$. Upper block: the component buildup. Lower block: the total against incidence, Eq. (11). The wing is absent by design—its profile drag is already inside the force tables.

Component	$S_{\text{wet}} [\text{m}^2]$	Re	C_f	FF	C_{D_0}
body	0.1422	8.41×10^5	0.00462	1.493	0.00521
duct	0.1246	1.56×10^5	0.00649	1.181	0.00507
friction total, incl. 3% excrescence allowance					0.01058
α [deg]	friction		crossflow		C_{D_0}
0	0.01058		0.00000		0.01058
4	0.01058		0.00006		0.01065
8	0.01058		0.00050		0.01108
12	0.01058		0.00166		0.01224
16	0.01058		0.00387		0.01445
20	0.01058		0.00739		0.01798
30	0.01058		0.02310		0.03369
45	0.01058		0.06535		0.07593
60	0.01058		0.12005		0.13063
90	0.01058		0.18483		0.19541

Table 16: Measured superposition error of the vane command modes: each simultaneous pair against the sum of its two single-mode increments, both taken from the neutral baseline at matched advance ratio. Magnitudes are $\|F\| + 10 \|M\|$ so that moment error is not swamped by force error. Pitch and yaw superpose; anything involving roll does not.

Pair	δ [deg]	J	combined	error	relative
pitch+yaw	+7.5	0.00	3.2015	0.0027	0.09%
	+7.5	0.20	4.4592	0.0021	0.05%
	+7.5	0.40	6.9067	0.0026	0.04%
pitch+roll	+7.5	0.00	1.9192	0.6182	32.21%
	+7.5	0.20	2.7324	0.7138	26.12%
	+7.5	0.40	4.5751	0.7666	16.76%
yaw+roll	+7.5	0.00	1.9114	0.6253	32.72%
	+7.5	0.20	2.7161	0.7136	26.27%
	+7.5	0.40	4.5637	0.7672	16.81%

2. **The failure is per-vane nonlinearity, not vane-to-vane interference.** This is what the pitch-plus-yaw result establishes: that case has all four vanes deflected simultaneously and still superposes to 0.05%, so the mutual influence between vanes is negligible at these deflections. What breaks superposition is only that a vane’s own response curves with its own angle.

The second conclusion prescribes the replacement structure. If the wrench is a sum of independent per-vane responses, each evaluated at that vane’s *total* commanded angle, then the correct tabulation is per-vane rather than per-mode—Eq. (36). That structure is nonlinear in each vane’s angle by construction, and it is also cheaper: seven tables in place of twenty-one, since one vane’s response serves all four positions under the cruciform’s rotational symmetry.

The replacement was then verified, not assumed. A separate sweep deflected each vane alone, and the per-vane summation model was used to predict every command in the superposition study—the three single modes and the three pairs—from those single-vane responses alone. Table 17 compares the two candidate structures on the same measurements.

Table 17: Worst-case prediction error of the two candidate buildups, over the three single modes and three simultaneous pairs at $\pm 7.5^\circ$ and $J \in \{0, 0.2, 0.4\}$.

Buildup	worst error	tables required
Sum of the three command modes	32.7%	21
Sum of the four per-vane responses, Eq. (36)	1.4%	7

The per-vane model predicts every case to within 1.4%, and that worst case is roll in hover—the same corner where the mode sum is worst by 33%. Pitch and yaw are predicted to 0.13%. The model that is simultaneously more accurate by a factor of twenty and cheaper by a factor of three is the one the file adopts, and adopting it cost one additional sweep of the driver that already existed.

10.3 Three conditions that do not converge, and what was measured about them

Of the 209 conditions in the generated map, 206 met the outer iteration’s convergence tolerance of 0.01. The three exceptions are all *positive* roll-mode commands at $J = 0.6$, with residuals of 0.027 to 0.066. Because a reader is entitled to know whether such cells are salvageable, both standard cures were tried and both failed:

Table 18: The non-converging corner. Neither a larger iteration budget nor a damped feedback gain resolves it.

δ_R [deg]	residual		thrust [N]		spread
	24 passes	48 passes	$\omega = 0.35$	$\omega = 0.15$	
−15	0.0061	0.0061	<i>converged in 3 passes</i>		
−10	0.0094	0.0094	<i>converged in 2 passes</i>		
−5	0.0066	0.0066	<i>converged in 2 passes</i>		
+5	0.0656	0.0656	4.8766	4.8764	0.004%
+10	0.0459	0.0465	4.5183	4.6779	3.5%
+15	0.0268	0.0278	4.1554	4.1580	0.06%

Doubling the iteration budget reproduces the residuals to three digits, and two of them worsen marginally—the signature of a stationary limit cycle in the block Gauss–Seidel alternation, not of slow convergence. Damping the vane-feedback relaxation from 0.35 to 0.15, the conventional remedy for an oscillating Gauss–Seidel iteration, likewise fails. The failure is moreover *sign-asymmetric*: negative roll commands converge in two or three passes at the same advance ratio. That asymmetry is physical rather than numerical—a common-mode deflection either adds to or opposes the residual swirl the vanes sit in, and only one sign drives the vane row toward the incidence at which its own solve is multivalued.

The useful outcome is not a fix but a measurement. Two independent iteration paths bound the exported value: thrust agrees to 0.004% and 0.06% at $\delta_R = +5^\circ$ and $+15^\circ$, so those cells are reproducible despite missing tolerance, while $\delta_R = +10^\circ$ differs by 3.5% and is genuinely path-dependent. The assembler carries all three as DAVE-ML uncertainty bounds—the mechanism the standard provides for precisely this—rather than as ordinary entries, and a consumer therefore sees a stated confidence rather than a false precision.

11 Verification

A generated file that nothing re-reads is the one artifact in a codebase that can rot silently. The model is therefore verified by its own contents.

11.1 Static shots

The standard provides for this directly: a `checkData` element carries one or more `staticShot` children, each an input vector (`checkInputs`), an optional dump of internal signals (`internalValues`), and an output vector with per-signal matching tolerances (`checkOutputs`) [2]. The model uses all three, and the `internalValues` dump is what makes a failing shot diagnosable rather than merely red—an intermediate coefficient that already disagrees localises the fault to a table, while agreement through to the final summation localises it to the buildup.

Every shot is generated from a solve; none is hand-written.

1. **Moment transfer.** One condition about the contract reference point and about a point displaced $+0.10$ m in x_{frd} ; the C_m difference must satisfy Eq. (3). Detects the frame-conversion failure described in Section 3.
2. **Aileron sign.** $\delta_a = +5^\circ$ at $\alpha = 4^\circ$; the solved C_l defines the sign, and `aileronDeg`’s variable definition cites this shot.
3. **Rate derivative.** $C_{l_p} = -0.452$, the value already pinned by the solver’s unit tests against the textbook figure for an $AR = 6$ wing.
4. **Vane modes.** Pure pitch, pure yaw and pure roll commands, each pinning the dominant components and recording the off-axis residuals at their solved values—these are physics, not zeros, and pinning them at zero would be a falsification.
5. **Propulsor reference.** The SciTech reference case, whose thrust already serves as that manuscript’s own consistency anchor.
6. **Part I cross-checks.** Attached-range C_L and C_m at several attitudes from the independent Part I matrix, at a tolerance that accommodates the viscous-versus-inviscid difference. This is the recorded justification for the overlap decision of Section 8.1.
7. **Summed wrench.** One fully populated condition—attitude, rates, aileron, vane commands, powered—evaluated through the buildup of Section 9 to dimensional forces and moments, pinning normalisation, summation and reference consistency end to end.

11.2 The evaluator

Verification runs through a small in-repo interpreter: it parses the file, performs multilinear interpolation on the gridded tables, evaluates the MathML buildup, and compares each static shot against its stated value. The alternative—depending on an external DAVE-ML implementation—was rejected on the same grounds that keep the rest of this codebase free of heavyweight dependencies, and because the MathML subset the file uses is under the author’s control precisely so that a compact evaluator suffices. The verifier runs in continuous integration, so the file cannot drift from the generators that produced it.

12 Generation pipeline

```
app/AeroMapExport.cpp -> models/data/aero-map.json
app/PropulsionMapExport.cpp -> models/data/propulsion-map.json
app/InductionMapExport.cpp -> models/data/coupling-map.json [blocked]
models/build-daveml.py -> models/AetherionFlightModel.dml
models/verify-daveml.py -> ctest suite
models/report/make-tables.py -> models/report/tables/*.tex
```

Each driver exports JSON independently and the assembler consumes the cached JSONs, so regenerating one map does not rerun the others. Rows are flushed as they complete, so an interrupted long sweep retains every finished condition. The JSON files are committed alongside the XML: the model and the data that produced it version together. This report’s data tables are generated from the same JSON by the same rule—no number in this document was typed by hand.

13 Declared limits

Collected here so that a consumer need not reconstruct them from the body of the report.

1. **Incompressible.** No Mach dependence exists in the generating methods. Validity $M < 0.3$. The rotor tip Mach number at the reference speed is 0.38, so the blades themselves sit at the edge of this assumption.
2. **Single Reynolds number.** All tables were generated at $V_\infty = 25$ m/s, $Re_n \approx 3 \times 10^5$.
3. **Axial inflow on the propulsor** (Section 8.3): exact in hover and axial climb, degrading through transition. No in-plane force or hub moment from disk incidence.
4. **Powered operation only.** The $\rho n^2 D^4$ group of Eq. (10) excludes $n \rightarrow 0$; windmilling and low-rotor-speed descent are outside the envelope.
5. **No interaction increments yet** (Section 8.4), pending an upstream-induction model. Until they exist, the airframe tables are power-off and the model underpredicts the separation delay the fan provides in transition.
6. **Parasite drag is a conceptual-design estimate** (Sections 4.1 and 10.1), carrying the conventional ± 10 –20% accuracy of a component buildup. It is moreover incomplete in one direction only: the duct’s own separated drag at incidence and any body-wake/wing interference are not modelled, and both omissions push the same way, so `aeroCD0` is a lower bound at high incidence.
7. **Post-stall values are cycle means** with uncertainty bounds, not steady states.
8. $C_{L\max}$ **is an upper bound:** bubble bursting is not modelled.

9. **No hysteresis.** The tables are the ascending- α branch. Static hysteresis is a known feature of this regime and a static gridded table cannot express it; representation is deferred to a dedicated study.
10. **Rate derivatives are tapered, not computed, past $\alpha = 20^\circ$** (Section 8.1).
11. **Vane mode superposition** is spot-checked, not tabulated pairwise.

14 Open items

- **Reconcile the variable identifiers against S-119 Annex A** (Section 1.1), the standard variable names. Where a standard name exists it should be adopted; where the namespacing requirement genuinely conflicts, the departure should be recorded in the file header rather than left implicit.
- Validate the assembled file against the DAVEfunc DTD as a build step, so that structural conformance is checked by a tool rather than by reading.
- Generate the airframe map (**aero***) and assemble the file.
- Extend the parasite estimate with the duct’s separated drag at incidence, the one contribution the slender-body form of Eq. (11) cannot supply.
- Build the upstream-induction model and unblock **coupling***.
- Inspect post-stall aileron effectiveness before shipping: the coupled solve should attenuate roll authority past separation, and a table that retained attached-flow authority at $\alpha = 40^\circ$ would model a simulator recovering from departures it should not.
- Measure and record vane-mode superposition error at soft-limit corners.
- Determine the sensitivity of the propulsor tables to the vane azimuth assumption.
- Confirm the rotor rotation sense against the aircraft’s actual installation; the generated map states the sense it used, and the contract states the axis but not the handedness of the reference speed about it.
- Acquire the validation anchors identified in [12] (isolated-propeller thrust and torque data; ducted-fan-with-vanes experiment or RANS comparison) before any table is used for certification-adjacent work.

References

- [1] ANSI/AIAA S-119-2011 (reaffirmed 2016), *Flight Dynamics Model Exchange Standard*, American Institute of Aeronautics and Astronautics, Reston, VA, approved 22 March 2011. doi:10.2514/4.867965. Annex A, *Standard Variable Names*; Annex B, *Dynamic Aerospace Vehicle Exchange Markup Language (DAVE-ML) Reference*.
- [2] AIAA Modeling and Simulation Technical Committee, Simulation Standards Subcommittee, “Dynamic Aerospace Vehicle Exchange Markup Language (DAVE-ML) Reference,” Version 2.0.1, 31 March 2011. <https://daveml.org/DTDs/prod/Ref/>
- [3] Jackson, E. B., and Hildreth, B. L., “Flight Dynamic Model Exchange using XML,” AIAA Modeling and Simulation Technologies Conference and Exhibit, Monterey, CA, AIAA Paper 2002-4482, August 2002. NASA NTRS 20030015249.

- [4] Jackson, E. B., and Hildreth, B. L., “Benefits to the Simulation Training Community of a New ANSI Standard for the Exchange of Aero Simulation Models,” NASA Langley Research Center, NTRS 20090041825, 2009.
- [5] “Status Report: ANSI/AIAA S-119-2011 Flight Dynamic Model Exchange Standard,” NASA Technical Reports Server, NTRS 20200004989, 2020.
- [6] ANSI/AIAA R-004-1992, *Recommended Practice for Atmospheric and Space Flight Vehicle Coordinate Systems*, American Institute of Aeronautics and Astronautics, Washington, DC, 1992.
- [7] ISO 1151-1:1988, *Flight Dynamics—Concepts, Quantities and Symbols—Part 1: Aircraft Motion Relative to the Air*, International Organization for Standardization, Geneva, 1988.
- [8] Bray, T., et al. (eds.), *Extensible Markup Language (XML) 1.1*, 2nd ed., W3C Recommendation, 2006.
- [9] Carlisle, D., Ion, P., Miner, R., and Poppelier, N. (eds.), *Mathematical Markup Language (MathML) Version 2.0*, 2nd ed., W3C Recommendation, 2003.
- [10] Tuncer, O., Üçler, Ç., and Güneş, A., “Aerodynamic Characterisation Across the Separation Boundary, Part I: Attachment Lines, the Separation March, and the Attached-Flow Envelope,” manuscript, 2026.
- [11] Tuncer, O., Üçler, Ç., and Güneş, A., “Aerodynamic Characterisation Across the Separation Boundary, Part II: Anchored Post-Stall Sections and the Attitude Map to 90° ,” manuscript, 2026.
- [12] Tuncer, O., Üçler, Ç., and Güneş, A., “Ducted-Fan Thrust Vectoring with Downstream Control Vanes: A Rotating-Frame Lattice Method with Two-Way Rotor–Vane Coupling,” AIAA SciTech Forum, manuscript, 2026.
- [13] Raymer, D. P., *Aircraft Design: A Conceptual Approach*, 6th ed., AIAA Education Series, American Institute of Aeronautics and Astronautics, Reston, VA, 2018.
- [14] Hoerner, S. F., *Fluid-Dynamic Drag*, published by the author, Bricktown, NJ, 1965.
- [15] Allen, H. J., and Perkins, E. W., “A Study of Effects of Viscosity on Flow over Slender Inclined Bodies of Revolution,” NACA Report 1048, 1951.
- [16] Jorgensen, L. H., “Prediction of Static Aerodynamic Characteristics for Slender Bodies Alone and with Lifting Surfaces to Very High Angles of Attack,” NASA TR R-474, 1977.
- [17] Viterna, L. A., and Corrigan, R. D., “Fixed Pitch Rotor Performance of Large Horizontal Axis Wind Turbines,” NASA Lewis Research Center, Cleveland, OH, 1982.